Slot API overview

When the GameWrapper is initialized, you can access the RGS features through `GameWrapper.prototype.rgs`.

The interface and the available methods depend on the `gameType` set during initialization. This document describes the methods and their responses for *'>slot'* games.

Note: To avoid blocked requests, it is recommended to check `GameWrapper.prototype.state` before calling RGS methods and only call them if the state is `active`. If the state is `paused`, the client should wait for a `game-resume` event before calling the desired method.

Common interfaces

RgsAccountData

```
{
  balance: number,
  currency: string
}
```

RgsCommonResponse

```
{
  game: RgsGameData,
  spin?: RgsSpinData,
  freespin?: RgsFreeSpinData,
}
```

RgsGameData

```
{
  nextAction: string,
  action: string,
  lines?: number,
  totalWin?: number,
  totalBet?: number,
  ticketId?: string,
  name?: string
}
```

RgsSpinData

```
{
  stops: number[],            // reel position stops
```

```
  grid: number[][],           // current symbol grid, as determined by stops
  secStops?: number[],        // secondary reel stops
  secGrid?: number[][],       // secondary reel grid
  totalBet: number,           // total bet
  wildMultiplier?: number,    // wild multiplier
  winAmount?: number,         // win amount of the spin
  betType: string,            // bet type
  lineCount: number,          // number of paylines selected by the player
  winCapped?: boolean,        // true if maximum win amount reached
  trigger?: RgsTriggerData    // free spin trigger data
  winnings?: RgsWinningsData  // winning pay line data
  feature?: any               // free-flow structure for unique game features
}
```

RgsFreeSpinData

```
{
  spinsAwarded: number          // number of awarded free spins
  spinsRemaining: number        // number of remaining free spins
  betType: string,              // bet type
  winAmount: number,            // win amount of the free spin
  stops: number[],              // reel position stops
  grid: number[][],             // current symbol grid, as determined by stops
  trigger?: RgsTriggerData[]    // free spin trigger data
  winnings?: RgsPaylineData     // winning pay line data
  retrigger?: RgsTriggerData[]  // free spin retrigger
  spinsRetrigger?: number       // number of new free spins awarded on retrigger
  spinsCapped?: boolean         // true if maximum amount of free spins reached
  wildListAwarded?: number[]    // new wild symbols, awarded for free spin game
}
```

RgsTriggerData

```
{
  symbol: string,     // symbol that triggered free spin
  offset: number[],   // reel offsets that represent the triggered pay line
}
```

RgsWinningsData

```
{
  symbol: string,     // winning symbol
  offset: number[],   // reel offsets that represent the triggered pay line
  payout: number,     // win amount for this pay line
  payline: number,    // payline index (-1 for scatter wins)
}
```

Methods

- `getAccount()`
- `getLineOptions()`
- `refresh()`
- `spin(lineCount: number, gameType: string, betPerLine: number)`
- `setFreeSpin(freeSpinType: string)`
- `freeSpin()`
- `close()`
- `getTicketData(ticketId?: string)`
- `setTicketData(data: string | object, ticketId?: string)`
- `getTicket(ticketId: string)`
- `sendCustomRequest(service: string, method: string, args: any[])`

`getAccount()`

Get user's current balance data including the currency. Calling this method will trigger a balance update throughout the whole GameWrapper.

Return value

- `Promise<RgsAccountData>` - a promise that resolves to `RgsAccountData` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getAccount()
  .then(function (data) {
    console.log('Current balance: ' + data.balance + ' ' + data.currency);
  }).catch(handleErrors);
```

Example response

```
{
  balance: 10000,
  currency: "GBP"
}
```

`getLineOptions()`

Get the list of allowed payline counts.

Return value

`number[]` - a list of available pay line counts.

Example

```
let lineOptions = wrapper.rgs.getLineOptions();
console.log('Allowed payline counts: ' + lineOptions);
```

Example result

```
[5, 10, 15, 20]
```

refresh()

Get game configuration data and the next pending action (*spin*, *setfreespin*, *freespin* or *close*). `refresh()` should be the first RGS API call after GameWrapper is initialized.

Usage example

```
wrapper.rgs.refresh()
  .then(function (data) { console.log('Received game data:', data.game) })
  .catch(handleErrors);
```

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Example responses

```
{
  game: {
    nextAction: "spin",
    action: "refresh",
  }
}
```

Example response in case of an unfinished game

```
{
  game: {
    nextaction: "close",
    action: "refresh",
    totalWin: 30,
    totalBet: 10,
    ticketId: "2d80860c-fa0d-11e8-bdb8-acbc327d774f",
    name: "fincore-golden-rooster"
  },
```

```
  spin: {
    stops: [17, 25, 1, 29, 34],
    grid: [
      [6, 5, 15],
      [8, 13, 15],
      [7, 1, 1],
      [8, 4, 13],
      [15, 2, 3]
    ],
    totalBet: 10,
    wildMultiplier: 3,
    winnings: [{
      symbol: 15,
      payout: 15,
      payline: 3,
      offset: [
        [0, 2],
        [1, 2],
        [2, 2]
      ]
    }, {
      symbol: 15,
      payout: 15,
      payline: 7,
      offset: [
        [0, 2],
        [1, 2],
        [2, 1]
      ]
    }],
    winAmount: 30
  }
}
```

`getGame()`

Get game data including stake, prize level and configuration data. `getGame()`.

Usage example

```
  wrapper.rgs.getGame()
    .then(function (data) { console.log('Available stakes: ' + data.ticketPrice) })
    .catch(handleErrors);
```

`spin(lineCount: number, gameType: string, betPerLine: number)`

Purchase and start a new spin.

Arguments

- `lineCount` - `{number}` number of selected paylines.
- `gameType` - `{string}` one of the game types provided by the game engine. E.g. 'S1D1', 'S2D1', etc.
- `bet` - `{number}` bet amount for this spin.

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.spin(10, 'S1D1', 1)
  .then(function (data) {
    console.log('Spin done. Next action is ' + data.game.nextAction);
  }).catch(handleErrors);
```

Example response

Example response with a win on two pay lines.


```
{
  game: {
    nextAction: "close",
    action: "spin",
    lines: 10,
    totalWin: 30,
    totalBet: 10,
    ticketId: "2d80860c-fa0d-11e8-bdb8-acbc327d774f",
    name: "fincore-golden-rooster",
    winCapped: false
  },
  spin: {
    stops: [17, 25, 1, 29, 34],
    grid: [
      [6, 5, 15],
      [8, 13, 15],
      [7, 1, 1],
      [8, 4, 13],
      [15, 2, 3]
```

```
    ],
    totalBet: 10,
    wildMultiplier: 3,
    winnings: [{
      symbol: 15,
      payout: 15,
      payline: 3,
      offset: [
        [0, 2],
        [1, 2],
        [2, 2]
      ]
    }, {
      symbol: 15,
      payout: 15,
      payline: 7,
      offset: [
        [0, 2],
        [1, 2],
        [2, 1]
      ]
    }],
    winAmount: 30
  }
}
```


Example response with free spins

Example spin response when a free spin is won.


```
{
  game: {
    nextAction: "freespin",
    action: "spin",
    lines: 10,
    totalWin: 1000,
    totalBet: 10,
    ticketId: "6ffac52a-fa11-11e8-8d06-acbc327d774f",
    name: "fincore-golden-rooster"
  },
  spin: {
    stops: [11, 8, 6, 2, 8],
    grid: [
      [17, 4, 6],
      [17, 5, 6],
      [15, 17, 13],
```

```
      [13, 17, 7],
      [2, 17, 8]
    ],
    totalBet: 10,
    wildMultiplier: 3,
    winnings: [{
      symbol: 17,
      payout: 1000,
      payline: -1,
      offset: [
        [0, 0],
        [1, 0],
        [2, 1],
        [3, 1],
        [4, 1]
      ]
    }],
    winAmount: 1000
  },
  freespin: {
    winAmount: 0,
    spinsAwarded: 15,
    spinsRemaining: 15,
    trigger: {
      symbol: 17,
      offset: [
        [0, 0],
        [1, 0],
        [2, 1],
        [3, 1],
        [4, 1]
      ]
    }
  }
}
```

`setFreeSpin(freeSpinType: string)`

Set the type of the next free spin game. Used for keno games where the player is offered the choice of winning different free games.

Argument

- `freeSpinType` - {string} - the desired free spin type. RGS checks the requested type and returns an error if the type isn't valid for the current game.

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.setFreeSpin('FG1')
  .then(function (data) {
    console.log('Free spin set. Call freeSpin() now');
  }).catch(handleErrors);
```

Example response

```
{
  game: {
    nextAction: 'freespin',
    action: 'setfreespin',
    totalWin: 625,
    totalBet: 12.5,
    ticketId: '0098e1ae-4a33-11e9-8225-acbc327d774f',
    name: 'fincore-golden-rooster'
  },
  freespin: {
    winAmount: 0,
    spinsAwarded: 6,
    spinsRemaining: 6,
    betType: 'FG1',
    wildListAwarded: [14, 1, 3]
  }
}
```

`freeSpin()`

Perform a free spin.

Argument

- `selectedNumbers` - {number[]} - an array of keno numbers selected by the player. This may be different selection from the one in the initial spin, but it must be at the same level (i.e. have the same amount of values).

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be

rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.freeSpin()
  .then(function (data) {
    console.log('Free Spin done. Next action is ' + data.game.nextAction);
  }).catch(handleErrors);
```

Example response


```
{
  game: {
    nextAction: 'freespin',
    action: 'freespin',
    lines: 25,
    totalWin: 2516,
    totalBet: 25,
    ticketId: '12a93ab4-1fe6-11e9-94f1-4a000244a410',
    name: 'fincore-golden-rooster'
  },
  freespin: {
    stops: [ 30, 20, 33, 6, 18 ],
    grid: [
      [ 13, 14, 2 ],
      [ 2, 7, 14 ],
      [ 14, 8, 3 ],
      [ 4, 13, 8 ],
      [ 5, 8, 17 ]
    ],
    winnings: [
      {
        symbol: 2,
        payout: 6,
        payline: 21,
        offset: [
          [ 0, 2 ],
          [ 1, 0 ]
        ]
      },
      {
        symbol: 14,
        payout: 10,
        payline: 23,
        offset: [
```

```
          [ 0, 1 ],
          [ 1, 2 ],
          [ 2, 0 ]
        ]
      }
    ],
    winAmount: 16,
    spinsAwarded: 15,
    spinsRemaining: 14,
    spinsCapped: true,
    winCapped: true
  }
}
```

`close()`

Finish the current game and close the ticket.

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.close()
  .then(function (data) { console.log('Ticket closed') })
  .catch(handleErrors);
```

Example response

```
{
  game: {
    nextAction: 'spin',
    action: 'close',
    totalWin: 2000,
    totalBet: 10,
    ticketId: '6ffac52a-fa11-11e8-8d06-acbc327d774f',
    name: 'fincore-golden-rooster'
  }
}
```

`getTicketData(ticketId?: string)`

Get data that was associated with a certain ticket.

If `ticketId` isn't specified, the wrapper will get data associated with the current ticket.

Argument

- `ticketId` - `{string}` ticket ID for which to get data. If `ticketId` isn't specified it will default to the current ticket ID.

Return value

- `Promise<string>` - a promise that resolves to a string-encoded data that was previously set with `setTicketData()`. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getTicketData('ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0')
  .then(function (data) {
    console.log('Ticket data: ' + data);
  }).catch(handleErrors);
```

Example response

```
'{"color": "red"}'
```

`setTicketData(data: string | object, ticketId?: string)`

Associate data with a ticket.

If `ticketId` isn't set it will default to the current ticket ID.

Arguments

- `data` - `{string | object}` the data to be associated with a ticket. May either be a JSON string or a serializable javascript object. The object will be stringified before sending the request to RGS.
- `ticketId` - `{string}` ticket ID for which to set data. If `ticketId` isn't specified it will default to the current ticket ID.

Return value

- `Promise<string>` - a promise that resolves to a string-encoded data that was set with `setTicketData()`. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

`getTicket(ticketId: string)`

Get ticket data for a previously played game. This is useful for getting historical ticket data, though this is usually handled by the GameWrapper and there is no

need for the client to call this method. For more details about history replay, see the Usage Guide.

Argument

- `ticketId` - {string} - ticket ID of a previously played game

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getTicket('ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0')
  .then(function (data) {
    console.log('Replaying game with ticket ID: ' + data.game.ticketId);
  }).catch(handleErrors)
```

Example response


```
{
  game: {
    nextAction: '',
    action: '',
    totalWin: 625,
    totalBet: 12.5,
    ticketId: '0098e1ae-4a33-11e9-8225-acbc327d774f',
    name: 'fincore-golden-rooster'
  },
  spin: {
    stops: [ 2, 17, 13, 4, 1 ],
    grid: [
      [ 12, 17, 12 ],
      [ 3, 17, 2 ],
      [ 13, 17, 4 ],
      [ 3, 17, 14 ],
      [ 12, 17, 1 ]
    ],
    totalBet: 12.5,
    wildMultiplier: 1,
    winnings: [
      {
        symbol: 17,
        payout: 625,
        payline: -1,
```

```
        offset: [
          [ 0, 1 ],
          [ 1, 1 ],
          [ 2, 1 ],
          [ 3, 1 ],
          [ 4, 1 ]
        ]
      }
    ],
    winAmount: 625,
    trigger: {
      symbol: 17,
      offset: [
        [ 0, 1 ],
        [ 1, 1 ],
        [ 2, 1 ],
        [ 3, 1 ],
        [ 4, 1 ]
      ]
    },
    betType: 'BG',
    lineCount: 25
  }
}
```

`sendCustomRequest(service: string, method: string, args: any[])`

Send a custom RGS request through the wrapper. This is useful when the client wants to use a feature which is supported by the RGS, but not implemented by the wrapper.

Should be used with caution. Best course of action is to ask first if this is the right way to implement required functionality. That is to avoid unnecessary debugging and delays.

Argument

- `service` - `{string}` RGS service name
- `method` - `{string}` RGS method name
- `args` - `{any[]}` array of arguments for the method. May be an empty array for no arguments.

Return value

- `Promise<any>` - a promise that resolves to the response object once a valid response is received. The exact response structure depends on the RGS method that is being called. In case of failure, the promise will be

rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
const gameId = /* some gameId related logic */;
wrapper.rgs.sendCustomRequest('SlotService', 'Refresh', [gameId])
  .then((data) => console.log('Refresh response: ', data))
  .catch(handleErrors);
```